

CS400M MINOR PROJECT

SmartKupon

SRS+SDS report

BY

Name and Roll No.

Abhinav Gupta (BTECH/25006/23)

Mehul Sharma (BTECH/25069/23)

Name of Supervisor

Dr. Piyush Gupta

(Assistant Professor)

Signature:



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BIRLA INSTITUTE OF TECHNOLOGY, MESRA

JAIPUR CAMPUS, JAIPUR

MO-2026

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Scope.....	3
1.3 Users of the System.....	4
1.4 Technology Stack	5
2. Software Requirements (SRS Part).....	7
2.1 Functional Requirements	7
2.2 Non-Functional Requirements	12
2.3 Constraints and Assumptions.....	13
3. System Design (SDS Part).....	14
3.1 System Architecture	14
3.2 Module Design	16
3.3 Database Design.....	19
3.4 User Interface Design	22
3.5 Data Flow Diagrams (DFD)	23
3.6 Sequence Diagrams.....	25
4. Conclusion	28
4.1 Summary	28
4.2 Benefits	28
4.3 Limitations.....	28
4.4 Future Scope	29
5. References	29

1. Introduction

1.1 Purpose

With the rapid growth of e-commerce, food delivery, digital payments, and online shopping platforms, users receive a large and constant stream of discount coupons and promotional offers through SMS messages and email. These offers are scattered across two disconnected channels, are mixed in with ordinary transactional and personal messages, and carry a silently ticking expiry date. As a result, a meaningful share of the discounts a user is entitled to are never noticed, are forgotten before checkout, or expire unused.

SmartKupon is proposed as a centralised mobile platform that removes the manual effort of hunting for coupons across SMS and Gmail. Instead of requiring the user to remember which app, which inbox, or which message held a given offer, the system authorises access to both sources, automatically extracts the coupon-relevant fields from unstructured message text, removes duplicates raised by repeated promotional blasts, and surfaces the result as a single, searchable, expiry-aware dashboard.

The purpose of this report is twofold. First, to specify — as a Software Requirements Specification (SRS) — the functional and non-functional behaviour SmartKupon must exhibit, grounded in the project synopsis submitted for this minor project. Second, to describe — as a Software Design Specification (SDS) — the architecture, module decomposition, database schema, interface design, and control flow through which those requirements are realised on the proposed Flutter / Django / PostgreSQL stack.

1.2 Scope

The scope of SmartKupon, as defined by the project proposal, includes the following capabilities:

- **Secure user authentication and account management** using Firebase Authentication, including Google Sign-In.
- **Authorised collection of coupon-related data** from Gmail and Android SMS, gated behind explicit user consent.
- **Extraction of structured coupon fields** — merchant name, coupon code, discount value, and expiry date — from unstructured message text using pattern matching and regex, with an optional NLP fallback.
- **Categorisation and centralised storage** of extracted coupon information in PostgreSQL.
- **Search and filtering** to help a user locate a relevant offer quickly.
- **Duplicate detection** so that a coupon broadcast repeatedly across several messages is tracked once, not many times.
- **Expiry tracking**, so a user can see at a glance which coupons are still valid and which are about to lapse.
- **A structured, mobile-first dashboard** presenting all of the above through a single Flutter application.

1.3 Users of the System

Actor	Description and Responsibilities	Access Level
Registered User	The primary and only human actor. Registers or signs in, authorizes Gmail and/or SMS access, views the coupon dashboard, searches and filters coupons, and manages individual coupon records (mark used / delete).	Full authenticated access to their own coupon and profile data only
System / Extraction Engine	A non-human, server-side actor. Periodically collects authorized source messages, applies the regex/NLP extraction pipeline, checks for duplicates, computes expiry status, and writes structured coupon records on the user's behalf.	Internal, backend-only; operates using tokens the user has explicitly granted

SmartKupon is modelled as a single-role consumer application. Authorisation is enforced entirely by data ownership — every coupon and source-message query is scoped to the authenticated user's identifier (see Section 2.2.2), so no cross-user access path exists even though only one role is defined.

1.4 Technology Stack

Layer	Technology	Purpose
Frontend (Mobile App)	Flutter (Dart)	Cross-platform mobile application development
Authentication	Firebase Authentication	User login, registration, and Google Sign-In
Backend Framework	Django	Business logic and server-side processing
API Development	Django REST Framework (DRF)	RESTful API creation and app-server communication
Database	PostgreSQL	Storage of user and coupon data
Email Integration	Gmail API	Fetching coupon-related emails with user authorization
SMS Integration	Android SMS Permissions	Reading promotional SMS messages on-device
Coupon Extraction Engine	Python, Regex	Extracting coupon codes, discounts, merchants, expiry dates
NLP Processing (Future Enhancement)	spaCy	Fallback extraction for irregularly formatted messages
Version Control	Git, GitHub	Source-code management and collaboration
Development Environment	Android Studio, VS Code	Application development and testing

1.4.1 Rationale for Key Choices

- **Flutter over native Android/iOS.** A single Dart codebase covers both target platforms with one UI implementation, which matters for a two-person team delivering a minor project on a fixed timeline. The trade-off, accepted explicitly in Section 2.3.1, is that Flutter does not change the underlying OS restriction on background SMS access — it inherits that limit rather than working around it.
- **Firebase Authentication over a hand-rolled auth system.** Registration, password storage, and Google Sign-In are security-sensitive and easy to implement incorrectly. Delegating them to Firebase removes an entire class of credential-handling bugs from the project's own code, leaving the Django backend to verify a signed ID token rather than manage passwords itself.
- **Django + DRF over a lighter framework.** The backend's job is not just serving JSON — it must also own the Gmail OAuth flow, a scheduled sync process, and the extraction pipeline. Django's batteries-included ORM, admin, and migration tooling reduce the boilerplate needed to get a correctly modelled multi-table schema (Section 3.3) into production quickly, while DRF supplies the serialisation and permission layers the REST API needs.
- **PostgreSQL over a document store.** Coupon records are relational by nature — a coupon belongs to exactly one user, references exactly one source message, and optionally joins to a merchant and one or more categories (Figure 3). Foreign-key constraints and unique indexes (e.g., the deduplication key in Section 3.2.5) are native to PostgreSQL and would have to be reimplemented in application code on a schema-less store.
- **Regex-first extraction, with spaCy as a fallback rather than the primary engine.** Promotional messages are short and formulaic (“USE CODE SAVE20”, “FLAT 50% OFF”, “valid till 12 Sep”), which a small, hand-curated regex library covers cheaply and predictably. Reaching for a full NLP model on every message would add latency and an unpredictable-failure surface for a problem regex already solves in the common case; spaCy is invoked only when the regex pass leaves fields missing (FR-3.3), keeping the common path fast and the tail case still recoverable.

2. Software Requirements (SRS Part)

This chapter specifies what SmartKupon must do (functional requirements) and the quality attributes it must exhibit (non-functional requirements). Functional requirements are grouped by module and given stable identifiers of the form FR-x.y so that each can be traced to the design elements in Chapter 3.

2.1 Functional Requirements

2.1.1 Authentication & Account Management (FR-1)

ID	Requirement	Description	Priority
FR-1.1	Registration & Login	The system shall allow a user to register and log in using Firebase Authentication (email/password).	High
FR-1.2	Google Sign-In	The system shall support Google Sign-In through Firebase Authentication to simplify onboarding and streamline subsequent Gmail authorization.	High
FR-1.3	Session Verification	The Django backend shall verify the Firebase-issued ID token on every authenticated request before granting access to a user's data.	High
FR-1.4	Logout	The system shall allow a user to log out, invalidating the client-side session.	Medium
FR-1.5	Profile Management	The system shall allow a user to view and update basic profile details associated with their account.	Low

2.1.2 Source Authorisation & Data Collection (FR-2)

ID	Requirement	Description	Priority
FR-2.1	Gmail Authorization	The system shall request explicit user consent (OAuth 2.0) before accessing Gmail messages, requesting read-only scope only.	High
FR-2.2	SMS Permission	The Flutter application shall request Android SMS read permission before accessing the device inbox, and shall remain usable in a Gmail-only mode if declined.	High
FR-2.3	Gmail Sync	The backend shall periodically fetch promotional/commercial-category emails for authorized users via the Gmail API.	High
FR-2.4	SMS Sync	The application shall read new promotional SMS messages on-device and forward the raw text to the backend for extraction.	High
FR-2.5	Minimal Retention	The system shall retain raw source-message text only as long as needed for extraction, discarding message bodies that yield no coupon data and persisting only coupon-relevant metadata thereafter.	Medium

2.1.3 Coupon Extraction Engine (FR-3)

ID	Requirement	Description	Priority
FR-3.1	Pattern-Based Extraction	The extraction engine shall apply a regex pattern library to identify coupon codes, discount values, merchant names, and expiry dates within unstructured message text.	High
FR-3.2	Date Normalization	The system shall normalize extracted expiry expressions (absolute dates and relative phrases such as “valid till Sunday”) into a standard date format.	High
FR-3.3	NLP Fallback	Where regex matching yields an incomplete record, the engine shall optionally apply spaCy-based named-entity extraction to recover missing fields.	Low
FR-3.4	Confidence Flagging	The system shall assign a confidence indicator to each extracted coupon and shall flag low-confidence records for the user to confirm or discard.	Medium

2.1.4 Coupon Management & Deduplication (FR-4)

ID	Requirement	Description	Priority
FR-4.1	Structured Storage	The system shall store each extracted coupon as a structured record (merchant, code, discount, expiry, source, category) scoped to the owning user.	High
FR-4.2	Duplicate Detection	The system shall check a new coupon candidate against existing records for the same user (matching code and merchant) before insertion, and shall merge or discard duplicates.	High
FR-4.3	Categorization	The system shall assign, or allow the user to assign, a category to each coupon for organized browsing.	Medium
FR-4.4	Manual Coupon Actions	The system shall allow a user to mark a coupon as used or to delete it manually.	Medium

2.1.5 Dashboard, Search & Filter (FR-5)

ID	Requirement	Description	Priority
FR-5.1	Coupon Dashboard	The system shall present all of a user's active coupons in a structured, scrollable dashboard.	High
FR-5.2	Search	The system shall allow a user to search coupons by merchant name, coupon code, or keyword.	High
FR-5.3	Filtering	The system shall allow a user to filter coupons by category, source (SMS/Gmail), and expiry window.	High
FR-5.4	Sorting	The system shall allow a user to sort coupons by expiry date or discount value.	Low

2.1.6 Expiry Tracking (FR-6)

ID	Requirement	Description	Priority
FR-6.1	Validity Status	The system shall compute and display each coupon's status (active, expiring soon, expired) based on its expiry date.	High
FR-6.2	Expiry Highlighting	The dashboard shall visually distinguish coupons expiring within a configurable window (e.g., three days).	Medium
FR-6.3	Expiry Notification	The system may notify a user in advance of a coupon's expiry; deferred to future scope (Section 4.4).	Low

2.1.7 API Communication (FR-7)

ID	Requirement	Description	Priority
FR-7.1	REST-Only Access	The Flutter application shall access all backend functionality exclusively through authenticated Django REST Framework endpoints; no coupon business logic shall execute on the client beyond presentation and on-device SMS reading.	High

2.2 Non-Functional Requirements

2.2.1 Performance

- Dashboard list and search/filter operations should return within **500 ms** for typical coupon volumes (≤ 500 records per user), excluding first-load network latency.
- Gmail and SMS sync shall run as background operations and must not block the Flutter UI thread.
- The extraction engine should process a single message well under one second under typical load.

2.2.2 Security & Privacy

- Gmail access uses OAuth 2.0 with read-only permissions; tokens are encrypted and stored securely on the backend.
- All communication between the Flutter app and Django backend uses HTTPS/TLS.
- The backend validates Firebase ID tokens on every request and rejects unauthorised access.
- Users can access only their own coupons and source data.
- Raw SMS and email content is discarded after coupon extraction; only coupon metadata is stored.
- SMS and Gmail permissions are requested with a clear explanation before authorisation.

2.2.3 Reliability

- A failure to parse one message shall not interrupt the sync of subsequent messages; failed extractions shall be logged and skipped rather than aborting the batch.
- Sync operations shall be idempotent — re-processing the same message shall not create duplicate coupons (see Section 3.2.5).
- The system should degrade gracefully when one source is unavailable, e.g. continuing SMS-based extraction if the Gmail API quota is exhausted.

2.2.4 Usability

- The mobile interface shall require no more than three taps from app launch to viewing an existing coupon's detail.
- Source-authorisation screens shall clearly explain what data is accessed and why, before the OS/OAuth permission prompt appears.
- Empty and loading states (no coupons yet, sync in progress) shall be explicitly designed rather than shown as a blank screen.

2.2.5 Scalability & Maintainability

- The Django / DRF / PostgreSQL architecture shall support horizontal scaling of the API tier as the user base grows.
- The regex pattern library shall be centralised and versioned separately from application logic, so new merchant message formats can be added without redeploying the app.
- The Flutter frontend, Django/DRF backend, extraction engine, and PostgreSQL data layer shall remain independently deployable modules.

2.3 Constraints and Assumptions

2.3.1 Constraints

- SMS reading is available only on **Android** — iOS restricts background/foreground SMS access — so SmartKupon's SMS-based extraction is an Android-only capability; iOS falls back to Gmail-only sync.
- Gmail data collection depends on user OAuth consent and is subject to Gmail API quota limits.
- Extraction accuracy depends on the regularity of promotional message formatting; highly irregular or heavily obfuscated messages may fail extraction and require manual entry.
- The system requires network connectivity for Gmail sync, authentication, and all backend communication; on-device SMS reading can occur offline, but syncing extracted data to the backend requires connectivity.
- Backend infrastructure must provide sufficient resources (a PostgreSQL instance and a scheduled sync process) to support periodic Gmail polling across all authorised users.

2.3.2 Assumptions

- Users receive a sufficient volume of coupon-bearing SMS/email traffic for a centralised aggregator to be valuable.
- Users are willing to grant SMS and/or Gmail permissions; the system provides a Gmail-only mode as a fallback for users unwilling to share SMS access.
- Promotional messages follow reasonably common phrasing conventions (“USE CODE”, “% OFF”, “valid till”) that a regex library can reasonably cover.
- Each SmartKupon account represents a single individual; shared or family coupon pools are out of scope.

3. System Design (SDS Part)

3.1 System Architecture

SmartKupon follows a layered client-server architecture. The Flutter mobile application is the sole presentation tier and is also where on-device SMS reading takes place, since the Android SMS permission is granted to the installed app, not to a server. Firebase Authentication forms an identity tier that both the client and the backend trust. Django, exposed through Django REST Framework, is the application/API tier: it verifies identity, orchestrates Gmail collection, invokes the coupon extraction engine, and is the only component that ever holds Gmail OAuth credentials or talks to PostgreSQL directly. PostgreSQL is the data tier. Figure 1 shows how these tiers connect.

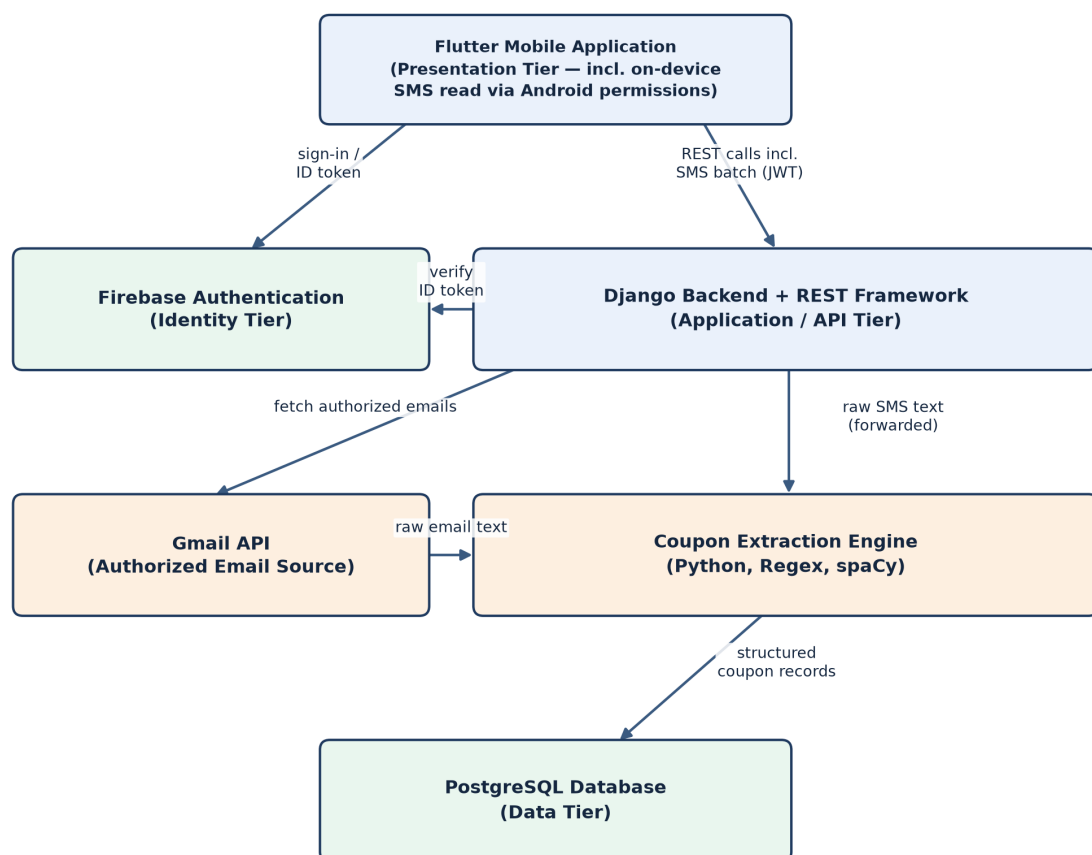


Figure 1 — SmartKupon layered system architecture.

3.1.1 Architecture Components

Component	Responsibility
Flutter Mobile Application	Presentation layer: authentication entry points, source-authorization screens, the coupon dashboard, search/filter UI, and on-device SMS reading via Android permissions.
Firebase Authentication	Handles registration, login, and Google Sign-In; issues short-lived ID tokens consumed by the backend.
Django Backend + REST Framework	Verifies ID tokens, exposes RESTful endpoints, orchestrates Gmail sync, and invokes the extraction engine.
Coupon Extraction Engine	A Python service (regex, optionally spaCy) that converts unstructured SMS/email text into structured coupon fields.
Gmail API	Authorized, read-only access to a user's promotional emails.
Android SMS Subsystem	On-device access to SMS messages, subject to explicit user permission.
PostgreSQL Database	Persistent storage for user accounts, coupon records, and source-message metadata.

3.2 Module Design

The system is decomposed into nine cohesive modules, split across the Flutter client and the Django backend.

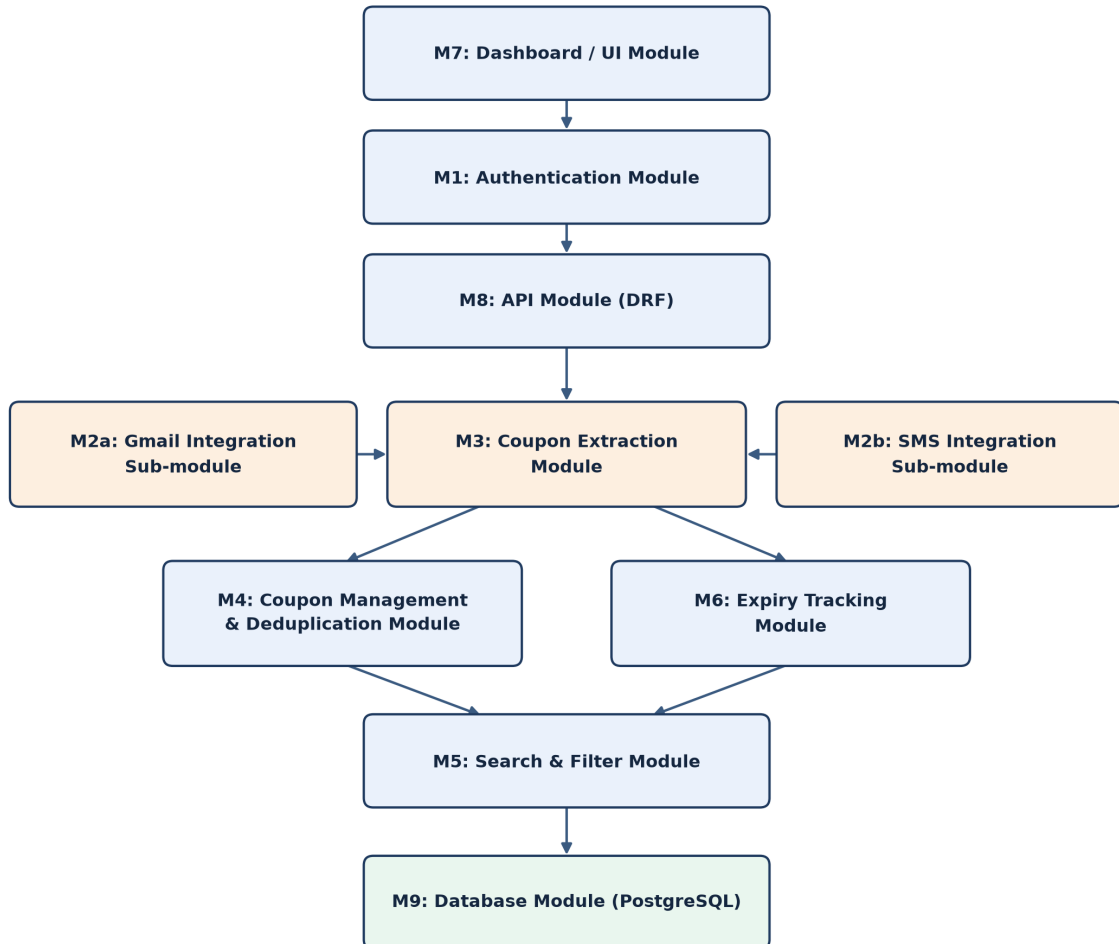


Figure 2 — Module interaction hierarchy.

3.2.1 Module Responsibilities

ID	Module	Responsibilities
M1	Authentication Module	Handles login, registration, Google Sign-In, and authenticated account access.
M2a / M2b	Gmail / SMS Integration Sub-modules	Handle authorized Gmail access and Android SMS-permission-based message collection respectively.
M3	Coupon Extraction Module	Processes source text and identifies merchant, coupon code, discount, and expiry information (Section 3.2.4).
M4	Coupon Management & Deduplication Module	Organizes extracted coupon records, assigns categories, and applies the duplicate-detection algorithm (Section 3.2.5).
M5	Search & Filter Module	Resolves search keywords and filter parameters into a query against the Coupon table.
M6	Expiry Tracking Module	Computes and annotates validity/ expiry status for coupon records.
M7	Dashboard / UI Module	Displays organized coupon information and handles user interaction within the Flutter application.
M8	API Module (DRF)	Provides REST endpoints (Section 3.2.3) for all frontend-backend communication.
M9	Database Module	Stores and retrieves user, source-message, and coupon data using PostgreSQL.

3.2.2 Module Interaction

After M1 authenticates the user, the user is routed through source authorisation, where M2a and/or M2b begin collecting raw message text. Both feed M3, which structures the text into candidate coupon fields. M4 checks each candidate against existing records via the deduplication key before writing to M9. M6 subsequently annotates stored records with their current validity status. M7 never touches M9 directly — it always goes through M8, which in turn calls M5 for search/filter requests and M6 for status information, keeping the Flutter client free of any direct database or extraction logic.

3.2.3 API Endpoint Specification

Method	Endpoint	Auth	Description
POST	/api/auth/session	Bootstraps	Verify a Firebase ID token and create/return the local user profile.
GET	/api/auth/me	Yes	Return the authenticated user's profile.
POST	/api/sources/gmail/authorize	Yes	Initiate the Gmail OAuth consent flow.
GET	/api/sources/gmail/callback	Yes	Handle the OAuth redirect and store encrypted tokens.
POST	/api/sources/sms/sync	Yes	Accept a batch of raw SMS text collected on-device for extraction.
POST	/api/sync/gmail/trigger	Yes	Manually trigger a Gmail sync cycle for the current user.
GET	/api/coupons	Yes	List the user's coupons (query params: search, category, source, status, sort).
GET	/api/coupons/{id}	Yes	Retrieve a single coupon's detail.
PATCH	/api/coupons/{id}	Yes	Update a coupon — e.g. mark used, change category.
DELETE	/api/coupons/{id}	Yes	Delete a coupon owned by the user.
GET	/api/coupons/expiring	Yes	List coupons expiring within a configurable window.

3.3 Database Design

The project proposal requires centralised storage of user and coupon data. The following logical schema represents that requirement; exact field types and migrations are finalised during backend development.

3.3.1 Main Entities

Entity	Purpose
User	A registered SmartKupon account, identified by its Firebase auth reference.
SourceMessage	The raw SMS/Gmail content a coupon was extracted from, kept only as long as needed for processing.
Coupon	A structured, deduplicated coupon record — the system's central entity.
Merchant	A normalized merchant name, referenced by many coupons.
Category	A user-facing grouping label, related to coupons many-to-many via CouponCategory.

3.3.2 Relationships

Figure 3 shows COUPON as the central entity: one User has many Coupon and SourceMessage records; one SourceMessage can yield one or more Coupon records (a single message occasionally advertises more than one code); one Merchant has many Coupon records; and Coupon relates to Category many-to-many through the CouponCategory junction table.

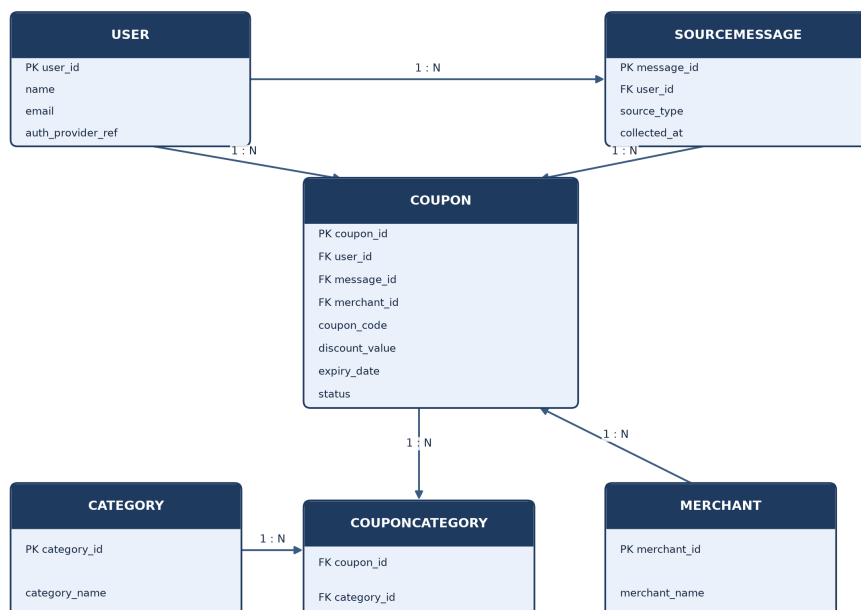


Figure 3 — Entity-relationship diagram for the SmartKupon logical schema.

3.3.3 Logical Schema

Table	Field	Type / Constraint	Notes
User	user_id	PK, UUID	Generated on first sign-in.
	name, email	String, NOT NULL	email unique, sourced from Firebase.
	auth_provider_ref	String, NOT NULL, unique	Firebase UID; links to the identity tier.
	created_at	Timestamp	Auto-set on insert.
SourceMessage	message_id	PK, UUID	
	user_id	FK -> User	Owning user.
	source_type	Enum(sms, gmail)	
	raw_ref / collected_at	Text / Timestamp	raw_ref cleared once extraction completes (FR-2.5).
Coupon	coupon_id	PK, UUID	
	user_id, message_id, merchant_id	FK	message_id nullable for manually added coupons.
	coupon_code, discount_value	String	As extracted / normalized.
	expiry_date	Date, nullable	Nullable when not extractable.
	status	Enum(active, expiring_soon, expired, used)	Maintained by M6 (Section 3.2.1).
	dedup_key	String, indexed (user_id, dedup_key)	normalize(merchant) + normalize(code); enforces FR-4.2.
Merchant	merchant_id / merchant_name	PK / String, unique	
Category	category_id / category_name	PK / String	
CouponCategory	coupon_id, category_id	Composite PK, FK x2	Junction table for the N:N relationship.

3.3.4 Design Rationale

- **Merchant is normalised into its own table** rather than stored as free text on Coupon, so that the same merchant extracted from many different messages ("Amazon", "amazon.in", "AMAZON") converges to one canonical row once the extraction engine's merchant-resolution step (Section 3.2.4) normalises it — this is what makes merchant-based search and filter (FR-5.2, FR-5.3) return complete results.
- **A composite (user_id, dedup_key) index drives duplicate detection** (Section 3.2.5) at the database layer rather than in application code alone, so the uniqueness guarantee holds even under concurrent sync jobs.
- **Category relates to Coupon many-to-many**, since a single offer (e.g. “20% off electronics”) can reasonably belong to more than one browsing category, and a category groups many coupons.
- **SourceMessage retains only a reference, not the full message body, after extraction** — this directly implements the minimal-retention requirement (FR-2.5, NFR Security & Privacy) rather than leaving it as an unenforced policy statement.

3.4 User Interface Design

3.4.1 Proposed Screens

Screen	Purpose
Splash / Onboarding	First-run introduction to what SmartKupon does and why it needs source access.
Login / Register	Secure account access, including Google Sign-In.
Source Authorization	Explains and requests Gmail OAuth consent and/or Android SMS permission.
Coupon Dashboard	Displays the user's active coupons in an organized, scrollable list.
Search / Filter	Keyword search plus category, source, and expiry-window filters.
Coupon Details	Merchant, code, discount, source, and expiry information for one coupon.
Expiring Soon View	A focused view highlighting coupons nearing their expiry window (FR-6.2).
Settings / Profile	Basic profile management and logout.

3.4.2 Navigation Flow

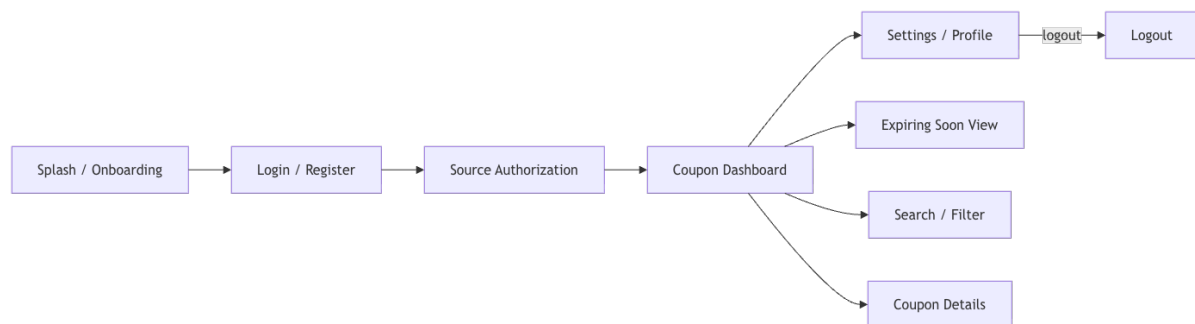


Figure 4 — Primary navigation flow across SmartKupon mobile app screens.

3.5 Data Flow Diagrams (DFD)

3.5.1 DFD Level 0

At Level 0, SmartKupon is treated as a single process. The Registered User exchanges credentials and search/filter requests for the coupon dashboard; Gmail and the Android SMS subsystem supply authorised message text; a single consolidated data store persists both user and coupon data.

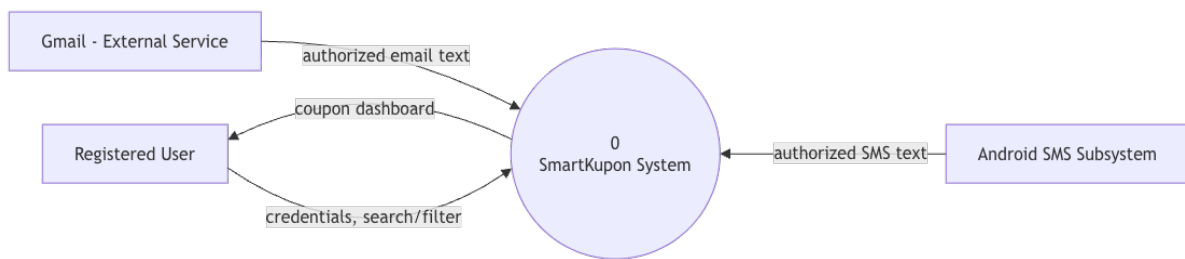


Figure 5 — DFD Level 0: SmartKupon as a single process.

3.5.2 DFD Level 1

Level 1 decomposes the system into six processes operating against three data stores, matching the module design in Section 3.2.

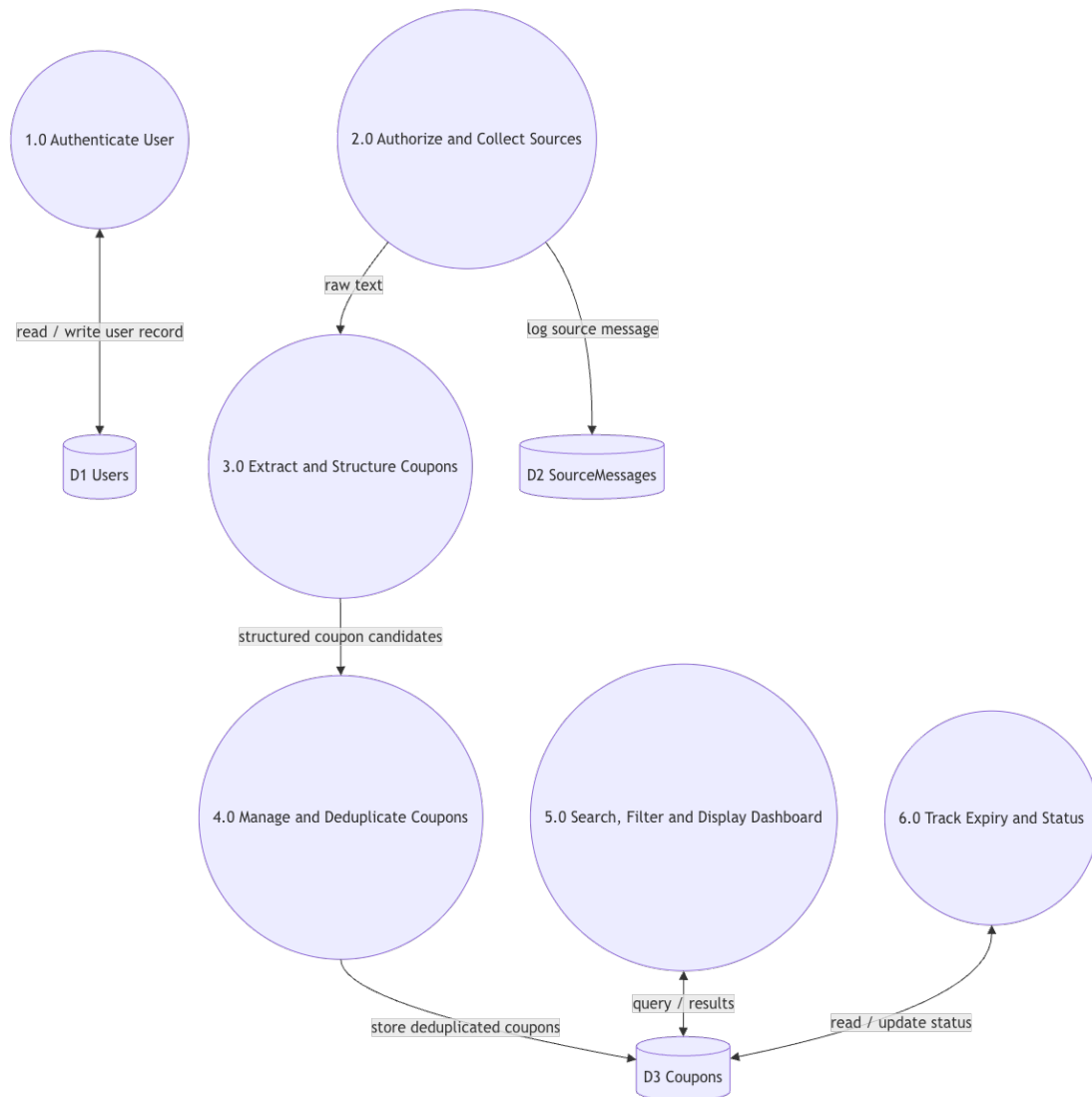


Figure 5 — DFD Level 1: process decomposition.

Process	Principal Data Flows
1.0 Authenticate User	In: credentials / ID token. Read/write: D1 Users.
2.0 Authorize & Collect Sources	Out: raw text to 3.0. Write: D2 SourceMessages (audit log).
3.0 Extract & Structure Coupons	In: raw text from 2.0. Out: structured coupon candidates to 4.0.
4.0 Manage & Deduplicate Coupons	In: candidates from 3.0. Write: D3 Coupons (Section 3.2.5).
5.0 Search, Filter & Display Dashboard	In: query parameters. Read: D3 Coupons.
6.0 Track Expiry & Status	Read/update: D3 Coupons (status annotation, Section 3.2.1).

3.6 Sequence Diagrams

3.6.1 Login & Source Authorisation Flow

The Flutter app never sees a password: Firebase issues an ID token after sign-in, the backend independently verifies that token before establishing a session, and Gmail authorization is requested as a separate, explicit step afterward (FR-1.1–FR-1.3, FR-2.1).

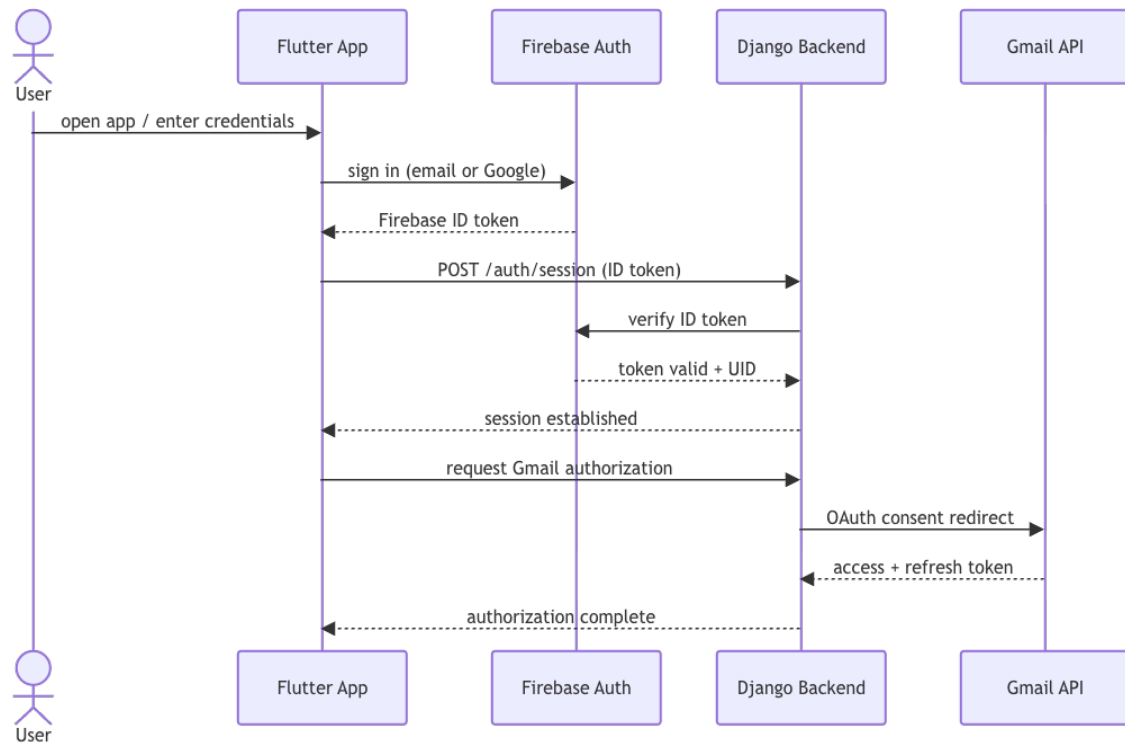


Figure 6 — Sequence diagram: user login and Gmail source authorization.

3.6.2 Coupon Collection, Extraction & Deduplication Flow

This is the core background pipeline of the system, implementing FR-2.3/FR-2.4, the extraction algorithm (Section 3.2.4), and the deduplication algorithm (Section 3.2.5) as one end-to-end flow from a new source message to a stored, duplicate-checked coupon record.

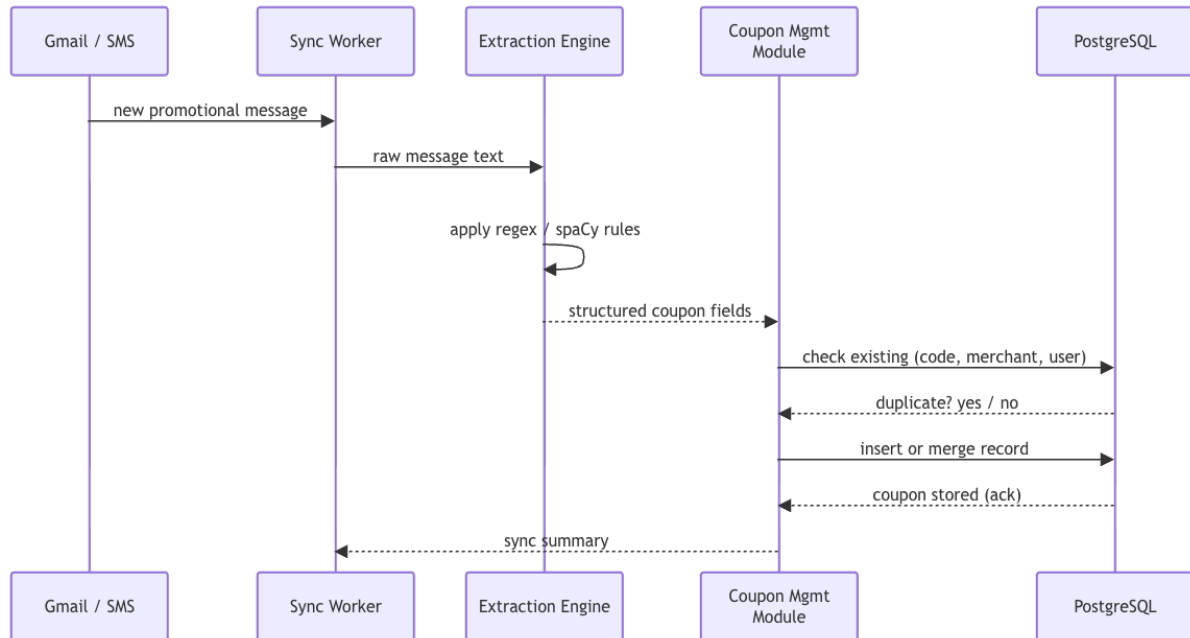


Figure 7 — Sequence diagram: coupon collection, extraction and duplicate-checked storage.

3.6.3 Search, Filter & Expiry Tracking Flow

Every dashboard read passes through the Expiry Tracking module so that validity status is computed at query time from the stored expiry_date rather than maintained as a separately drifting field, which keeps FR-6.1 correct even for coupons that were never re-synced after insertion.

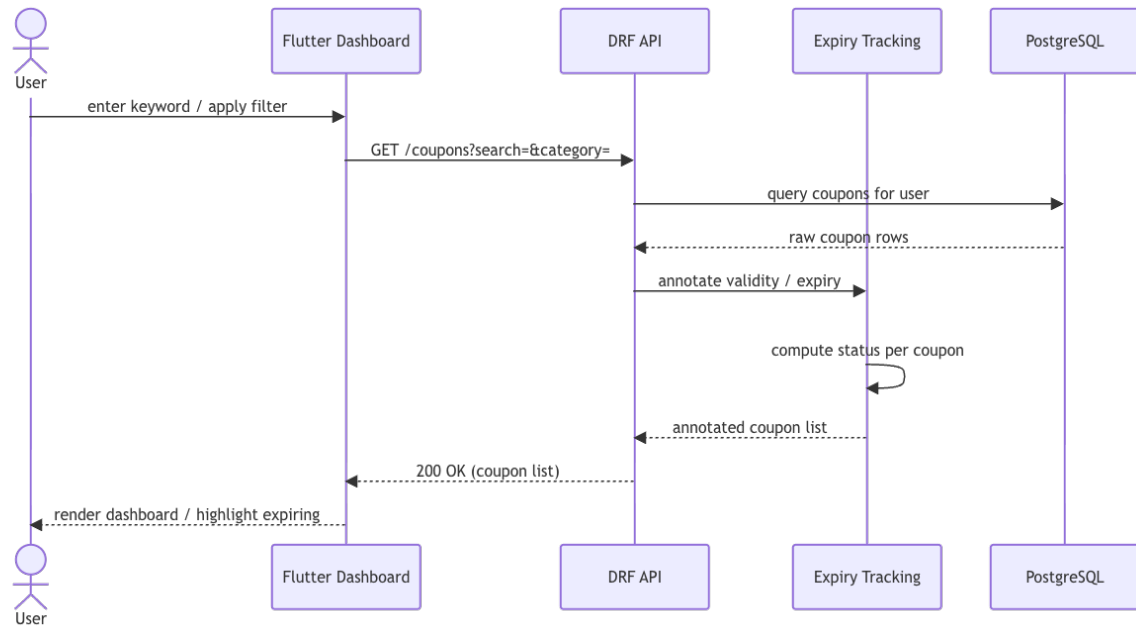


Figure 8 — Sequence diagram: search, filtering and expiry-status display on the dashboard.

4. Conclusion

4.1 Summary

SmartKupon proposes a centralised platform for a problem that is easy to underestimate: not the absence of discounts, but their invisibility once scattered across SMS and email. The system's engineering substance lies in the pipeline connecting the two authorised sources to one trustworthy dashboard — a regex-first extraction engine with an NLP fallback for the cases plain patterns miss (Section 3.2.4), a per-user deduplication key that collapses repeated promotional blasts into one tracked offer (Section 3.2.5), and an expiry-status computation that runs at query time so the dashboard is never showing stale validity information (Section 3.6.3). The architecture separates the mobile interface, identity, application/API, extraction, and storage into independently deployable tiers (Figure 1), giving the two-person team a structure that a minor-project timeline can realistically build and test module by module.

4.2 Benefits

- Centralised aggregation of coupon information from both SMS and email, in one dashboard.
- Reduced manual effort in locating promotional offers before they expire.
- Structured storage of merchant, coupon code, discount, and expiry information.
- Fast search and filtering across all of a user's available coupons.
- Duplicate coupon detection, keeping the dashboard signal-dense rather than repetitive.
- Explicit coupon validity and expiry tracking, so a user always knows what is still usable.
- A single, authenticated mobile interface for everything the proposal defines as in scope.

4.3 Limitations

- **SMS extraction is Android-only.** iOS does not permit third-party background SMS access, so an iOS build necessarily operates in Gmail-only mode (Section 2.3.1) — this is a platform restriction, not an implementation gap.
- **Extraction accuracy is bounded by message regularity.** A regex-first engine, even with the spaCy fallback, will still miss coupons buried in unusually worded or heavily templated marketing copy; the confidence-flagging mechanism (FR-3.4) surfaces these cases for manual review rather than silently dropping or silently trusting them.
- **No cross-user or community coupon sharing.** The system only ever surfaces coupons the individual user's own SMS/Gmail already contains; it does not discover offers the user was never sent (see Future Scope, 4.4).
- **Deduplication is per-user and code/merchant-based,** so two genuinely different but identically worded promotional codes from the same merchant sent at different times could, in rare cases, be merged when they should not be — an accepted trade-off for keeping the dashboard uncluttered.

4.4 Future Scope

- **Internet-based coupon discovery**, extending beyond a user's own inbox to publicly listed offers, as identified in the original project proposal.
- **AI-powered coupon recommendations**, surfacing offers relevant to a user's likely spending categories rather than a flat chronological or expiry-sorted list.
- **Proactive expiry notifications** (FR-6.3), closing the loop on Section 2.2.4's usability goals with a push notification a configurable number of days before a coupon lapses.
- **A forwarded-email fallback for iOS**, letting a user forward promotional emails to a dedicated address as a workaround for the platform's SMS restriction.
- **A browser extension**, applying a stored coupon code automatically at checkout on supported merchant sites.

5. References

References are listed in IEEE style.

- [1] A. Gupta and M. Sharma, "SmartKupon – Unified Coupon Discovery and Management Platform," Project Synopsis, CS400M Minor Project, Dept. of Computer Science & Engineering, Birla Institute of Technology, Mesra, Jaipur Campus, 2026.
- [2] Department of Computer Science & Engineering, Birla Institute of Technology, Mesra, Jaipur Campus, "CS400M Minor Project – Web-based SRS+SDS Report Format/Template," supplied file Webbased_SRS_SDS_Project_report_format_final.docx, 2026.
- [3] Google LLC, "Flutter Documentation," Flutter, 2025. [Online]. Available: <https://docs.flutter.dev/>
- [4] Google LLC, "Firebase Authentication Documentation," Firebase, 2025. [Online]. Available: <https://firebase.google.com/docs/auth>
- [5] Django Software Foundation, "Django Documentation," 2025. [Online]. Available: <https://docs.djangoproject.com/>
- [6] Encode OSS Ltd., "Django REST Framework Documentation," 2025. [Online]. Available: <https://www.django-rest-framework.org/>
- [7] PostgreSQL Global Development Group, "PostgreSQL Documentation," 2025. [Online]. Available: <https://www.postgresql.org/docs/>
- [8] Google LLC, "Gmail API Overview," Google for Developers, 2025. [Online]. Available: <https://developers.google.com/gmail/api>
- [9] Android Open Source Project, "SMS Permissions — Permissions on Android," Android Developers, 2025. [Online]. Available: <https://developer.android.com/guide/topics/permissions/overview>
- [10] Explosion AI, "spaCy — Industrial-Strength Natural Language Processing," 2025. [Online]. Available: <https://spacy.io/>
- [11] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.
- [12] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson Education, 2016.
- [13] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.